Practical - 5

Practical Title: Registration Form with Frontend Validation and User-Friendly Error Handling

Problem Definition: Create a student registration form with HTML5 input types and JavaScript validation for name, email, mobile number, password, confirm password, course, year, gender, and terms acceptance. Use Regular Expression for validations.

A. Theory

1. Are correct input types and attributes used?

Answer:

Yes, suitable HTML5 input types and attributes are used for all fields:

• type="text" is used for Full Name with minlength and maxlength attributes to ensure proper length.

• type="email" is used for Email Address to open email-specific keyboards on phones and check standard format.

• type="tel" is used for Mobile Number to show number keypad on mobile devices and restrict input length to 10 digits.

• type="password" is used for Password and Confirm Password to hide characters on screen, along with an eye icon toggle button to show or hide the password.

• select dropdown is used for Course to let students pick from available degree programs.

• type="radio" inside fieldset is used for Academic Year and Gender where only one option should be chosen.

• type="checkbox" is used for Terms Acceptance to ensure the student accepts college rules before submitting.

• novalidate attribute is added to the form tag so custom JavaScript validation handles all checks and displays styled error messages.

Reference: https://developer.mozilla.org/en-US/docs/Web/HTML/Element/input and https://www.w3schools.com/html/html_forms.asp

2. Are validation errors displayed near the relevant fields?

Answer:

Yes, validation errors are placed right below each input field:

• Each field has an error message span tag directly underneath it. When the input is invalid, red error text explains what needs to be fixed.

• The input border turns red when invalid, and turns green with a checkmark once the input is valid.

• We use input and change event listeners to give real-time feedback as the user types, instead of waiting only for form submission.

• When the form is submitted with errors, the page automatically moves focus to the first invalid field so the student can fix it right away.

Reference: https://www.w3schools.com/js/js_validation.asp and https://web.dev/learn/forms/

3. Is password strength checked?

Answer:

Yes, password strength is checked dynamically as the user types:

• A JavaScript function checks five conditions: at least 8 characters, at least one uppercase letter, at least one lowercase letter, at least one digit, and at least one special symbol.

• A visual progress meter updates between Weak (red), Medium (orange), and Strong (green) based on how many criteria are met.

• A simple checklist below the password field turns from a cross to a green checkmark when each rule is fulfilled.

• A regular expression is used on form submit to ensure only passwords meeting all rules can be submitted.

Reference: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Regular_expressions

4. Is the form accessible using keyboard and screen-reader-friendly labels?

Answer:

Yes, the form is designed to be accessible for all users:

• Every input element has a matching label connected using the for and id attributes. Clicking a label focuses the input field.

• Radio buttons for Academic Year and Gender are wrapped inside fieldset and legend tags so screen readers read the section name before reading the options.

• The aria-describedby attribute connects input fields to their error messages so screen readers announce the error when an invalid input is selected.

• All controls including inputs, buttons, checkboxes, and the CAPTCHA refresh button can be navigated using the Tab key on the keyboard.

Reference: https://web.dev/learn/accessibility/forms/

B. Implementation / Output / Result

The student registration form was built for the StudentHub portal using clean HTML5, CSS3, and JavaScript:

• Input Validation: Name checks alphabetic characters using /^[A-Za-z\s]{3,50}$/. Email checks valid email format using /^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/. Mobile number checks 10 digits starting with 6-9 using /^[6-9]\d{9}$/.

• Password Features: Includes a password strength bar, instant requirement checkmarks, a toggle to show/hide text, and matching validation between password and confirm password.

• Selection Controls: Dropdown for Degree Course, radio buttons for Academic Year and Gender, and terms acceptance checkbox.

• Additional Features: Simple numeric/canvas CAPTCHA verification to stop automated bots, real-time feedback with green/red borders, dark mode support, and a summary popup showing submitted details.

C. Evidence of Work (Screenshots)

Figure 5.1: Student Registration Form Overview

Description: The complete registration form with all required student fields, clean layout, and matching StudentHub theme.

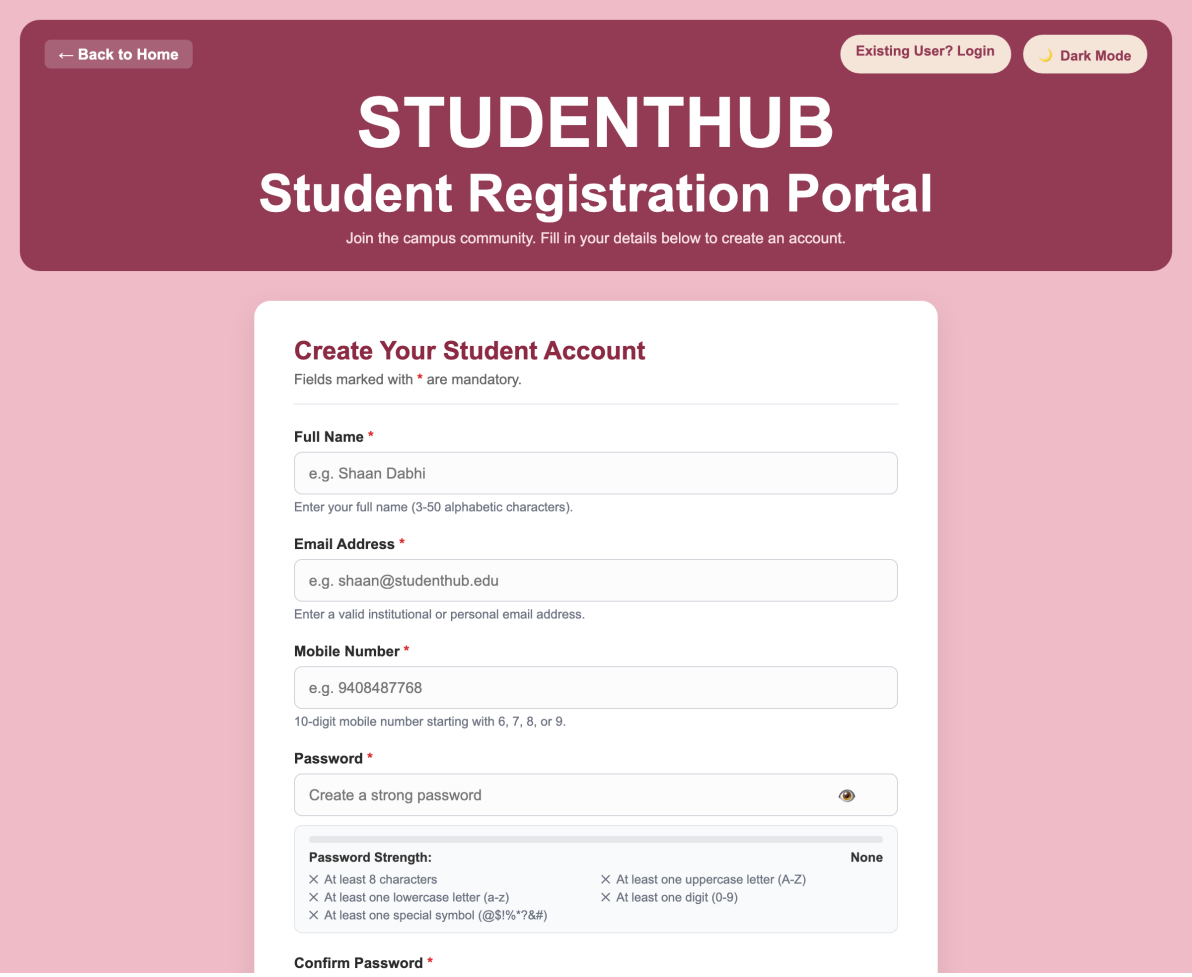


Figure 5.2: Field Validation and Real-Time Error Messages

Description: Inline error messages in red and red border highlights when invalid data is entered.

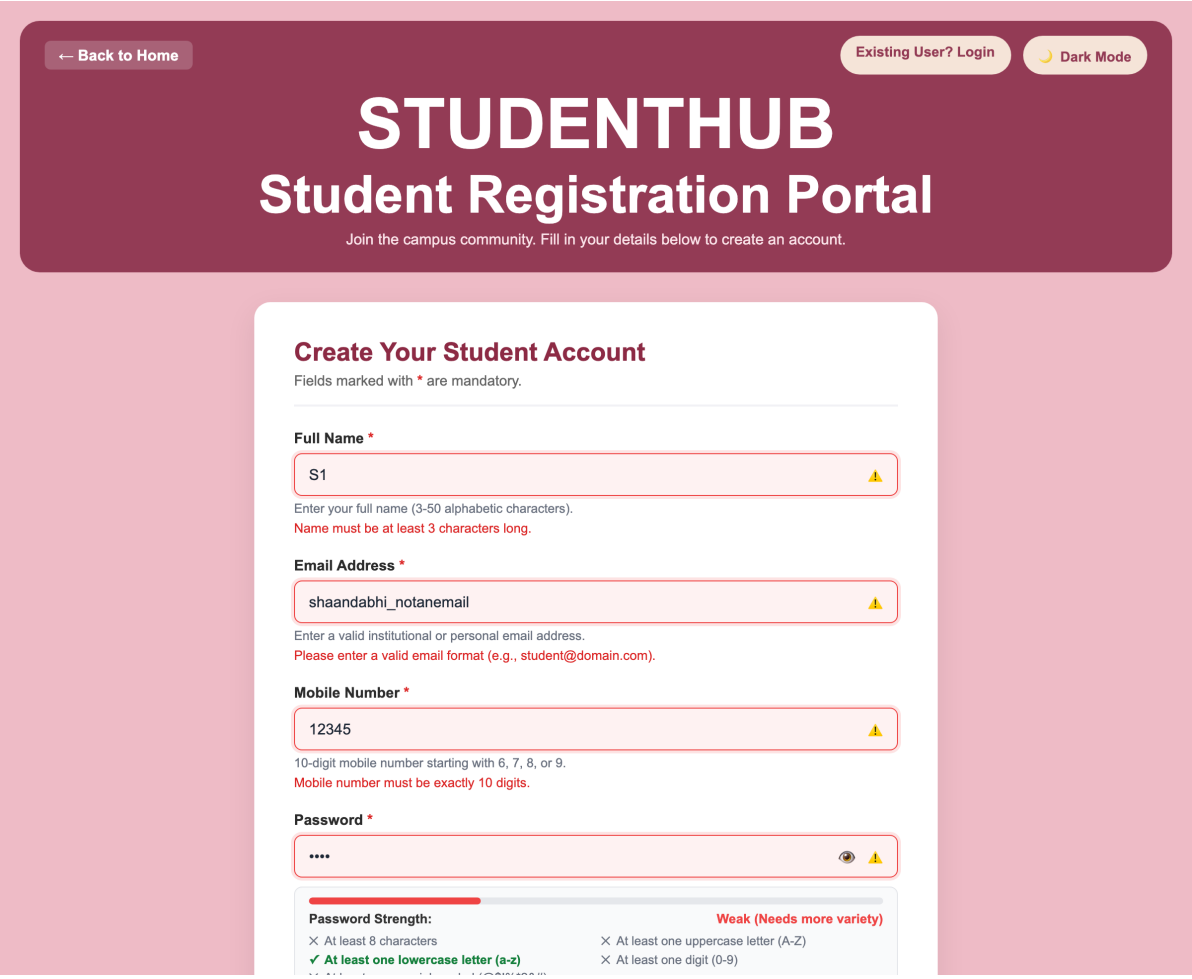


Figure 5.3: Password Strength Meter and Requirement Checklist

Description: The visual strength meter showing color feedback and live checkmarks for password rules.

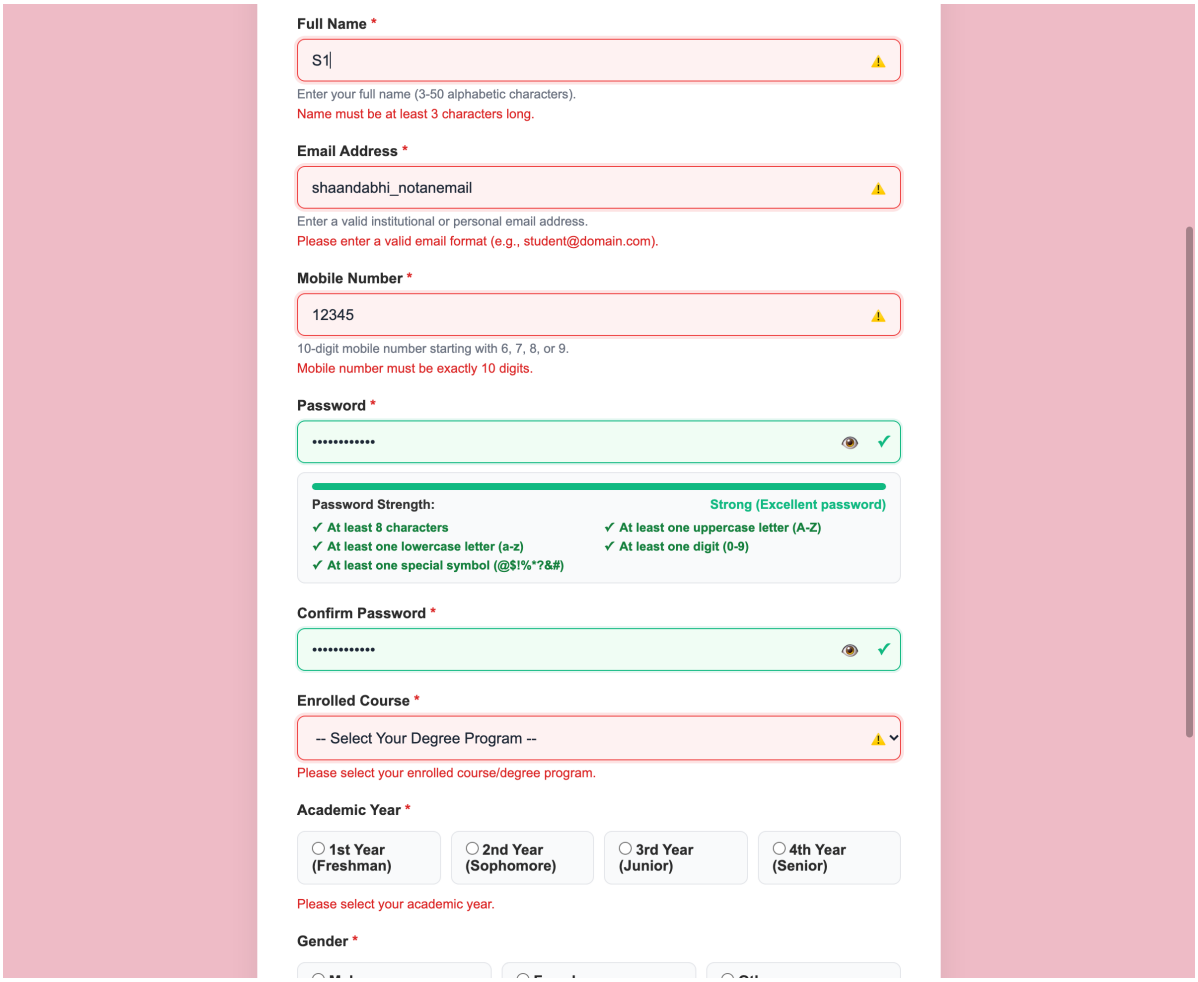


Figure 5.4: Successful Registration and Submission Summary

Description: The confirmation modal showing the student data summary and successfully cleared inputs.

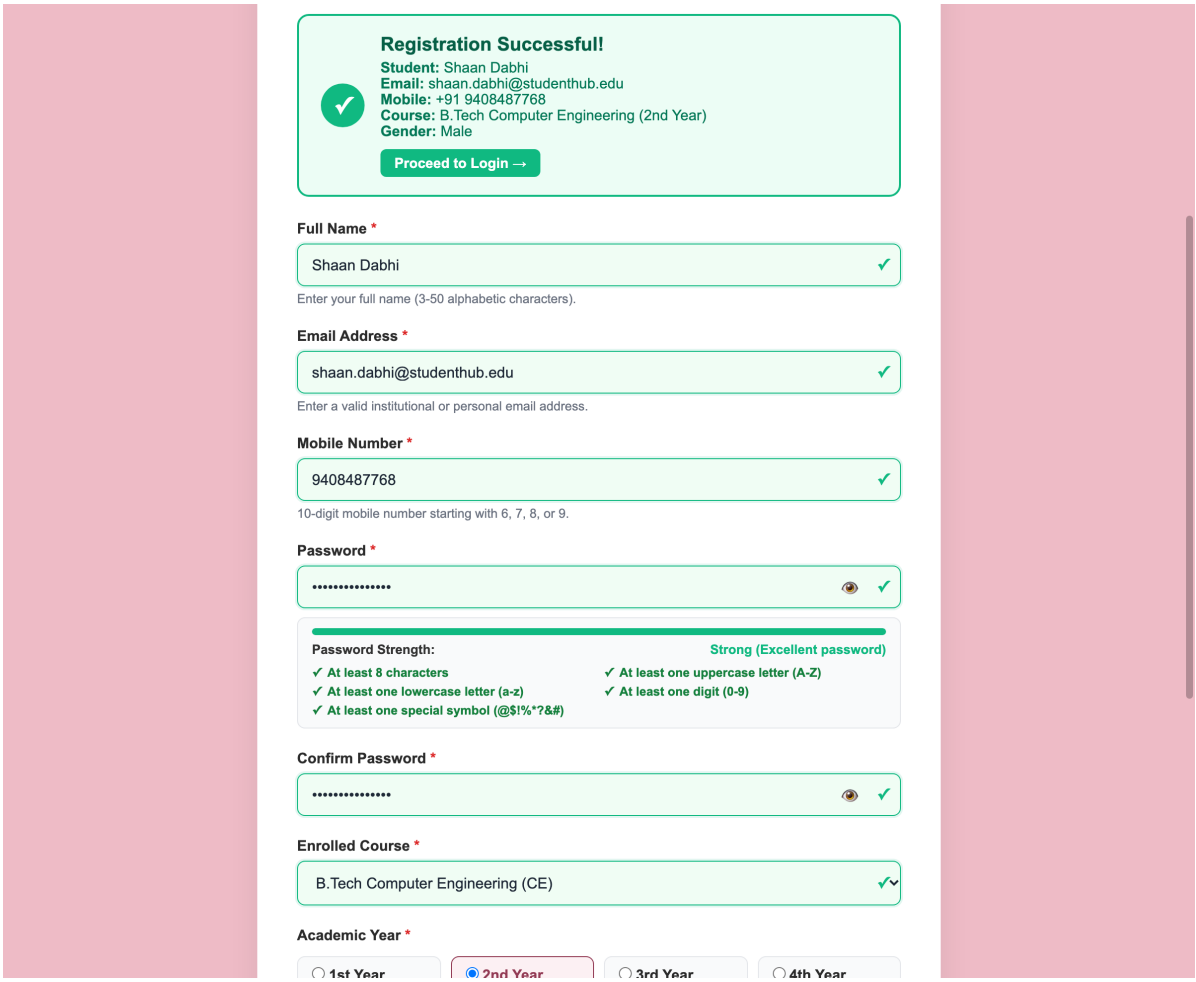
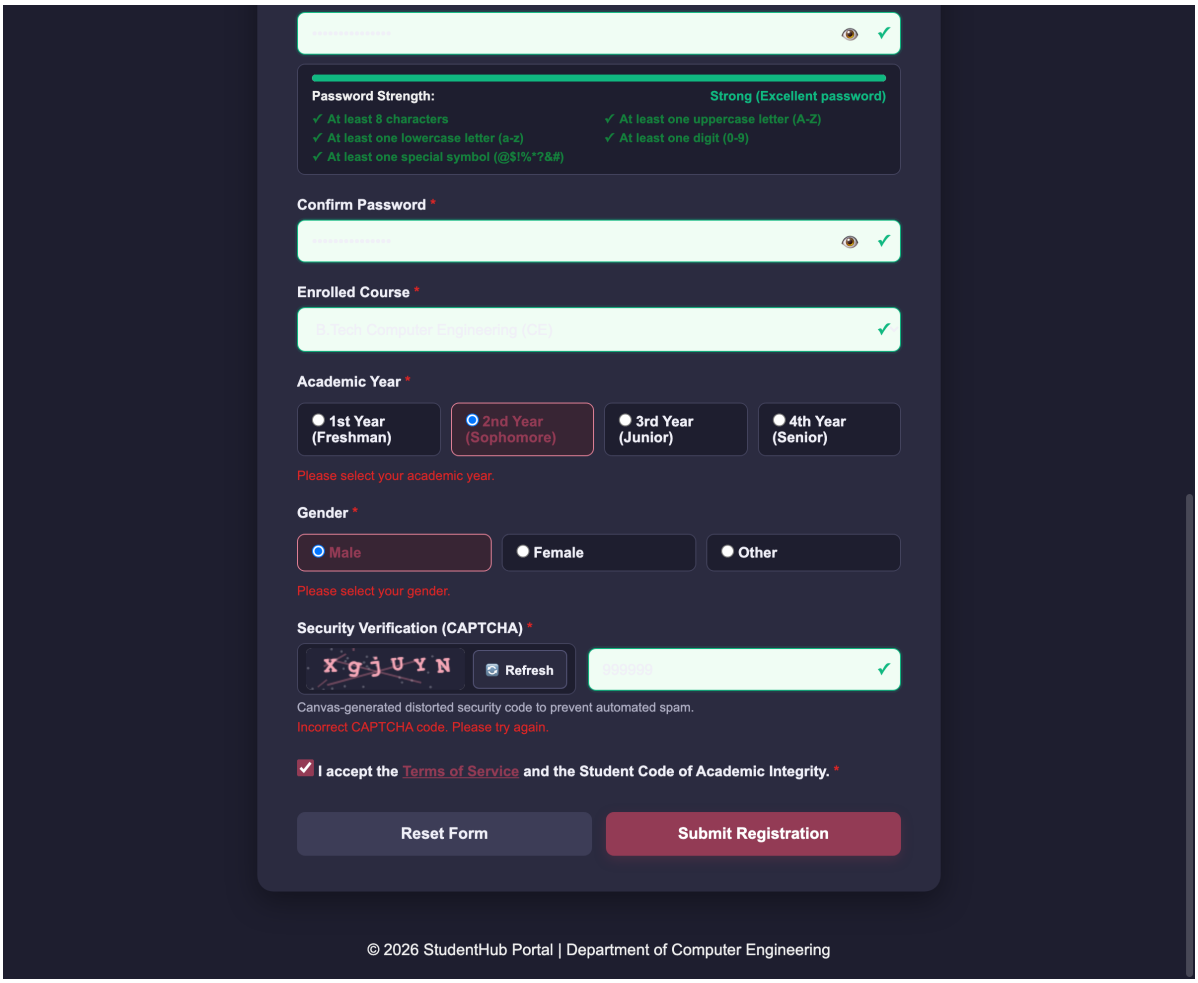


Figure 5.5: Dark Mode Theme and Bot Protection Verification

Description: The registration form running in dark mode theme with active CAPTCHA verification.



D. GitHub Repository Link:

https://github.com/shaandabhi1578/student-hub

E. Challenges Faced:

1. Regular Expressions for Names: Simple regex stopped names with spaces or hyphens. Updated the regex to /^[A-Za-z]+([\s-][A-Za-z]+)*$/ to accept full names properly.

2. Immediate Password Match Check: Confirm password check needed to update dynamically whenever either the password or confirm password field was edited.

3. Real-Time Feedback vs Submit State: Balancing instant typing feedback so users are not annoyed before finishing typing by validating on both input and blur events.

4. Mobile Friendly Keyboards: Used type="tel" and type="email" so mobile phones automatically switch to the right keyboard layout.

F. Learning Outcomes:

From this practical, I learned how to create accessible web forms with client-side JavaScript validation:

• Selecting correct HTML5 input types for better mobile and desktop user experience.

• Writing regular expressions to check names, emails, phone numbers, and strong passwords.

• Giving immediate visual feedback using dynamic border colors and inline error messages.

• Building password meters and visibility toggle buttons to improve form usability.

• Making forms accessible using proper labels, fieldsets, and keyboard navigation.

Faculty Remarks:

Criteria	Remarks / Grade
Understanding of Concepts & Validation Logic	
Timely Completion & Code Cleanliness	